

Tr_11 – Esquema Relacional

Documento de Referencia: **Esquema Relacional**

Proyecto: Aplicación **“No Te Cortes”**

Sector: **Servicios de Estética e Imagen Personal (CNAE 9602)**

1. Justificación del Diseño

Para completar la arquitectura de **"No Te Cortes"**, el esquema relacional traduce el diagrama Entidad-Relación a una estructura de tablas normalizada. He aplicado las reglas de transformación para asegurar que no haya redundancia y que la integridad referencial se mantenga, garantizando la consistencia de los datos en los flujos de Cliente, Administrador y Empleado.

2. Definición de Tablas y Atributos

A continuación, se detallan las tablas que componen el sistema. (Notación: **PK** = Primary Key, FK = Foreign Key)

TABLA: USUARIOS

Campo	Tipo de Dato	Restricción
id_user	String (UID)	PK , Not Null
nombre	String	Not Null
apellidos	String	Not Null
email	String	Unique, Not Null
telefono	String	Not Null
direccion	String	
ciudad	String	
rol	String	Check (Client, Admin, Emp)
fcmToken	String	Para notificaciones Push

Campo	Tipo de Dato	Restricción
notificaciones	Boolean	Default True
activo	Boolean	Default True
createdAt	Long	Timestamp de registro

TABLA: SERVICIOS

Campo	Tipo de Dato	Restricción
id_servicio	String	PK , Not Null
nombre	String	Not Null
precio	Decimal (Double)	Check (> 0) , Not Null
duracion	Int	Minutos, Not Null

TABLA: CITAS

Campo	Tipo de Dato	Restricción
id_cita	String	PK, Not Null
fecha	Date	Not Null
hora	String	Not Null
estado	String	Default 'Pendiente'
userId	String	FK, Not Null (USUARIOS.id_user)
id_servicio	String	FK, Not Null (SERVICIOS.id_servicio)

TABLA: PROMOCIONES

Campo	Tipo de Dato	Restricción
id_promo	String	PK , Not Null
descripcion	String	Not Null
precio	Decimal	Not Null, Check (>0)
dirigidoA	String	Check (Todos, Nuevos, VIP)
fechaInicio	Date	Formato ISO/Date
fechaFin	Date	Formato ISO/Date
activa	Boolean	Default True
id_admin	String	FK (USUARIOS.id_user)

TABLA: FECHAS_BLOQUEADAS

Campo	Tipo de Dato	Restricción
fecha	Date	PK , Not Null
motivo	String	Not Null
id_admin	String	FK (USUARIOS.id_user)
createdAt	Long	Timestamp

TABLA: PASSWORD_RESETS

Campo	Tipo de Dato	Restricción
id_reset	String	PK, Not Null
userId	String	FK (USUARIOS.id_user)
token	String	Not Null
expiracion	DateTime	Not Null

3. Restricciones de Integridad y Lógica de Transformación

Se han definido las siguientes reglas:

1. **Transformación de Relaciones 1:N:** Las relaciones "Realiza" (Usuario-Cita) y "Solicita" (Cita-Servicio) se han resuelto mediante la propagación de claves. La tabla **CITAS** absorbe userId e id_servicio como llaves foráneas.
2. **Integridad Referencial:** * No se permite la creación de una cita para un usuario inexistente (Constraint FK_userId).
 - Si un servicio se elimina del catálogo, el sistema debe impedir el borrado si existen citas pendientes vinculadas (RESTRICT).
3. **Integridad de Dominio:** Se asegura que el campo precio sea siempre positivo y que el rol coincida con los perfiles definidos en los bocetos (Tr_07).

4. Implementación Mediante Script SQL (DDL)

Como valor añadido, se presenta el código para materializar este esquema:

```
CREATE TABLE USUARIOS (  
id_user VARCHAR(50) PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
apellidos VARCHAR(100) NOT NULL,  
email VARCHAR(100) UNIQUE NOT NULL,  
telefono VARCHAR(20) NOT NULL,  
direccion VARCHAR(255),  
ciudad VARCHAR(100),
```

```

rol VARCHAR(20) NOT NULL CHECK (rol IN ('Client', 'Admin', 'Emp')),
fcmToken VARCHAR(255),
notificaciones BOOLEAN DEFAULT TRUE,
activo BOOLEAN DEFAULT TRUE, createdAt BIGINT
);

CREATE TABLE SERVICIOS (
    id_servicio VARCHAR(50) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) NOT NULL CHECK (precio > 0),
    duracion INT NOT NULL CHECK (duracion > 0)
);

CREATE TABLE CITAS (
    id_cita VARCHAR(50) PRIMARY KEY,
    fecha DATE NOT NULL,
    hora VARCHAR(10) NOT NULL,
    estado VARCHAR(20) DEFAULT 'Pendiente',
    userId VARCHAR(50) NOT NULL,
    id_servicio VARCHAR(50) NOT NULL,
    FOREIGN KEY (userId) REFERENCES USUARIOS(id_user) ON DELETE CASCADE,
    FOREIGN KEY (id_servicio) REFERENCES SERVICIOS(id_servicio) ON DELETE
    RESTRICT
);

CREATE TABLE PROMOCIONES (
    id_promo VARCHAR(50) PRIMARY KEY,
    descripcion TEXT NOT NULL,
    precio DECIMAL(10,2) NOT NULL CHECK (precio > 0),
    dirigidoA VARCHAR(50),
    fechaInicio DATE,
    activa BOOLEAN DEFAULT TRUE,
    id_admin VARCHAR(50),
    FOREIGN KEY (id_admin) REFERENCES USUARIOS(id_user)
);

```

```

CREATE TABLE FECHAS_BLOQUEADAS (
fecha DATE PRIMARY KEY,
motivo VARCHAR(255) NOT NULL,
id_admin VARCHAR(50), createdAt BIGINT,
FOREIGN KEY (id_admin) REFERENCES USUARIOS(id_user)
);

CREATE TABLE PASSWORD_RESETS (
id_reset VARCHAR(50) PRIMARY KEY,
userId VARCHAR(50),
token VARCHAR(100) NOT NULL,
expiracion DATETIME NOT NULL,
FOREIGN KEY (userId) REFERENCES
USUARIOS(id_user) ON DELETE CASCADE
);

```

5. Consideraciones para la implementación en Cloud Firestore (NoSQL)

Aunque el diseño lógico se ha formalizado mediante un Esquema Relacional y scripts SQL para garantizar la integridad de las reglas de negocio, la implementación física se llevará a cabo en **Cloud Firestore**. Para adaptar este modelo relacional a la arquitectura NoSQL, se han definido las siguientes estrategias:

- **Desnormalización y Rendimiento:** Se aplicará desnormalización en la colección citas, duplicando campos como nombreServicio o nombreCliente. Esto evita realizar múltiples consultas (*joins* manuales) y garantiza una carga instantánea en la interfaz del Administrador y el Cliente.
- **Jerarquía de Subcolecciones:** Para optimizar la seguridad y el acceso, las citas se estructurarán como una **subcolección** dentro del documento de cada usuario. Esto permite aplicar Reglas de Seguridad de Firebase más granulares.
- **Mapeo de Tipos de Datos:** * Los campos definidos como Long (createdAt) y Date se implementarán mediante el tipo **Timestamp** nativo de Firestore para permitir filtrados y ordenación cronológica eficiente.
 - El id_user será el **UID** generado por Firebase Authentication, vinculando la identidad del usuario con su registro en la base de datos de forma unívoca.
- **Integridad en el Lado del Cliente/Funciones:** Al no existir restricciones de clave foránea nativas en NoSQL, las reglas de integridad (como el RESTRICT al borrar servicios) se gestionarán mediante **Cloud Functions** y validaciones en la lógica de la aplicación.